Manulife - Level of Effort (LoE): Package Risk & EOL Automation

Prisma Cloud Automation Services

Palo Alto Networks

Dec 23, 2025

Executive Summary

Total Estimated Effort: 16 Days

Primary Objective: Develop a custom automation solution to provide a unified view of package risks for Manulife's Cloud Workload Protection (CWP) environment. This solution will aggregate **AppSec Findings** (Vulnerabilities/CVEs) and enrich them with **End-of-Life (EOL)** and **End-of-Support (EOS)** status data.

Critical Assumption: The solution will leverage public/open-source feeds for lifecycle data (EOL/EOS). To ensure performance, a deduplication layer will be used to process unique [Package + Version] pairs before merging them back with asset-specific vulnerability findings.

Introduction

This document outlines the Level of Effort (LoE) required to establish a centralized Package Risk and Lifecycle Monitoring solution.

While Prisma Cloud CWP provides robust visibility into AppSec findings (vulnerabilities and compliance), there is a requirement to extend this context to include software lifecycle status—specifically **End-of-Life (EOL)** and **End-of-Support (EOS)**. This automation acts as a bridge, programmatically extracting both the Software Bill of Materials (SBOM) and existing AppSec findings from the platform, and correlating them with external lifecycle data sources.

This document provides a detailed overview of the planned activities, task breakdowns, and time estimates associated with this implementation.

Justification

Unified Risk Visibility: Manulife requires a single pane of glass that reveals not only which packages have active vulnerabilities (CVEs) but also which packages are operating beyond their vendor support dates (EOL/EOS). Currently, these datasets exist separately or are incomplete; this automation bridges that gap.

Proactive Lifecycle Management: By identifying EOS dates alongside active AppSec findings, operational teams can prioritize upgrades for components that are both vulnerable and unsupported, preventing "forever day" exploits.

Optimized Data Processing: Given the scale of container environments, thousands of images share common base layers. This solution implements intelligent deduplication to analyze package lifecycles efficiently, reducing API overhead and execution time.

Architecture Overview

The architecture involves a dual-stream **Extraction Engine** (fetching SBOMs and AppSec Findings), a **Deduplication & Enrichment Layer** (for EOL/EOS lookup), and a **Correlation Engine** to merge the datasets.

1. **Ingestion:** Automation logic queries the Prisma Cloud CWP API to retrieve:
    - **SBOM Details:** Inventory of all packages per asset.
    - **AppSec Findings:** Existing vulnerability data and package risks detected by Prisma.
2. **Deduplication Layer:** A processing step that identifies unique [Package Name + Version] pairs across the estate to optimize external API queries.
3. **Enrichment:** Logic to query external public repositories for the unique package list to append **EOL** and **EOS** dates.
4. **Correlation & Output:** The enriched lifecycle data is merged with the AppSec findings. The final dataset is formatted specifically for Manulife's stakeholder reporting needs, including potential integration with visualization tools like **Grafana**.

Hardware & Resource Requirements:

- **Serverless:**

  Memory: 2–4 GB per function
  CPU: Proportional to memory allocation (typically 1–2 vCPU)
  Execution timeout: 10–15 minutes per function
  Ephemeral storage: ≥ 1 GB
  Concurrency: Intentionally limited to avoid Prisma API throttling

- **Instance-Based:**

  CPU: 4–8 vCPU
  Memory: 16–32 GB RAM
  Disk: 100 GB SSD
  Network: High-throughput, low-latency access to Prisma APIs

- **URL Whitelisting:**

  https://endoflife.date/
  Prisma Cloud instance URL

The estimated time to complete this automation work is **16 days**.

Planned Activities and Estimations

Analysis & Data Strategy (3 Days)

- **API & Data Mapping:** Analyze the Prisma Cloud CWP API response structures for both SBOM exports and AppSec/Vulnerability findings.
- **Schema Definition:** Define the unified data schema that will hold both security data (CVE ID, Severity, CVSS) and lifecycle data (EOL Date, EOS Date, Days Remaining).
- **Deduplication Strategy:** Finalize the logic to ensure accurate correlation between unique packages and the multiple images/hosts that contain them.

Dual-Stream Extraction Logic (4 Days)

- **SBOM Extraction:** Develop the script to iterate through CWP assets (Images, Hosts) to build the package inventory.
- **AppSec Findings Extraction:** Implement logic to pull detailed vulnerability and risk profiles for the corresponding assets.
- **Optimization:** Implement pagination and efficient memory handling to process large volumes of finding data (which is significantly larger than just SBOM data) without timeouts.

Deduplication & Enrichment Engine (EOL/EOS) (4 Days)

- **Deduplication:** Implement logic to parse the raw data and generate a unique list of packages to avoid redundant lookups.
- **Lifecycle Enrichment:** Develop the interface to query public data feeds for the unique package list to retrieve:
    - **End-of-Life (EOL):** When the software stops receiving updates.
    - **End-of-Support (EOS):** When the vendor stops providing support.
- **Error Handling:** Implement robust handling for "unknown" packages or API rate limits on external feeds.

Correlation, Output Generation & Visualization Strategy (4 Days)

- **Stakeholder Consultation:** Work directly with Manulife stakeholders to define the exact presentation format required for executive and operational reporting.
- **Grafana Integration Assessment:** Evaluate requirements for Grafana integration.
    - If required, design the output structure (JSON/InfluxDB line protocol) to ensure seamless ingestion by Grafana.
    - Implement the necessary data transformation logic to support dashboard visualization (e.g., time-series tagging).
- **Data Merging:** Develop the core logic to map the enriched EOL/EOS data *back* to the specific AppSec findings.
- **Report Generation:** Build the final output generation module:
    - **CSV:** Formatted for easy filtering by "EOS Status" or "Severity".
    - **JSON:** Structured for the agreed-upon visualization or SIEM tool.

Testing & Documentation (1 Day)

- **Validation:** Conduct end-to-end testing with a representative dataset to validate that AppSec findings are correctly aligned with the calculated EOL/EOS dates.
- **As-Built Documentation:** Create a technical guide detailing script usage, output formats, and the interpretation of the merged risk fields.
- **Handoff:** Formal code and documentation transfer to the Manulife team.

Scope Assumptions

Data Source: The project relies on publicly available EOL/EOS data feeds. Coverage is strictly limited to what these open sources provide.

AppSec Data: The automation extracts *existing* findings from Prisma Cloud. It does not perform new scans; it aggregates data already present in the platform.

Execution: The script is designed to be executed in a customer-provisioned environment (e.g., local runner, VM, or Cloud Function).

Out of Scope

Commercial Data Feeds: Procurement of paid/private EOL intelligence feeds.

Complex Dashboard Hosting: While the automation will provide data formatted for Grafana (if required), the hosting, maintenance, and licensing of the Grafana infrastructure itself are out of scope.

Remediation: The tool identifies risks but does not automatically upgrade packages or patch images.

Identified Risks & Dependencies

Data Volume (AppSec Findings): extracting vulnerability data for a large environment can result in massive datasets. The script will require sufficient memory on the execution runner.

Public Data Consistency: Open source feeds may vary in naming conventions. Best-effort matching logic will be applied, but 100% matching of EOL/EOS dates cannot be guaranteed without curated commercial feeds.